

Hexaxia DAS: A Deterministic, Human-and-Machine-Readable Document Addressing Standard

Aaron Lamb

Founder | Visionary Architect, Hexaxia Technologies

hexaxia.tech · linkedin.com/in/aaron-lamb-hex

Version 1.0, a position paper. Finalized 2026-06-10 (first draft 2026-05-29). This is a statement of position, not a validated result. The measured evidence is preliminary (see Section 5, which states its limits), and the central resilience thesis still waits on the at-scale experiment described in Section 7. Thesis and claim are kept distinct from evidence throughout.

DAS is a working name. The asset is the standard, not the label.

1. Abstract

Hexaxia DAS (Document Addressing Standard) is a deterministic, human-and-machine-readable addressing layer for document corpora. It lays out a corpus so that one identifier does two jobs at once: it is a permanent, hierarchical, decimal address a person can read, dictate, and cite, and one an agent can load as a single map and resolve without embeddings. The thesis is that a corpus laid out this way stays navigable without RAG, at organizational scale, in the regime where free-form wikis and Johnny.Decimal both fall apart.

What DAS leads with is not retrieval speed. It is the deterministic floor. A corpus addressed this way stays navigable when the retrieval layer is offline, air-gapped, degraded, or just not trusted for the task at hand. Inside a larger agentic stack, DAS is the floor under the probabilistic machinery. It is the layer you have to be able to lose and still keep working. The address costs almost nothing to maintain, and it outlives everything above it.

2. The problem

Three failure modes motivate DAS, and they compound as agentic workflows multiply documents.

Ragless at scale does not hold. A pattern has emerged of running a knowledge base with no retrieval layer at all. This is the Karpathy “LLM Wiki” idea, the Obsidian-plus-agent setup, agent-synthesized markdown you navigate directly. It works beautifully for a personal corpus of a few hundred curated pages. It does not survive organizational scale. Once a corpus grows into thousands of heterogeneous documents written by many hands and agents, navigating it directly with no index and no convention turns into guesswork. There is no canonical map, and no fast way to rule out the regions that do not matter.

Johnny.Decimal has a two-level ceiling. JD is the closest prior art and the lineage DAS owns up to: permanent numeric addresses, human-readable, no infrastructure. But JD was built for one person, and it deliberately caps at two levels: one area, one category. A company with a dozen clients, several service lines, internal departments, and years of piled-up documents does not fit into ten areas of ten categories in any meaningful way. JD also has no machine-readable description of its own address space and no enforcement, so at organizational scale the convention drifts from author to author and quietly rots.

Retrieval is not memory, and an index is not a floor. Vector search finds things that sound like the query. It has no stable citation primitive, it cannot be grepped or audited offline, and it means nothing the moment its index is gone. A corpus whose only way in is an embedding service is unusable without that service. That is fine for a convenience layer and not fine for a system of record.

3. What DAS is

DAS is a layout standard for a filesystem corpus. The normative detail lives in [docs/spec.md](#). This section is a short orientation.

Addresses. Every node carries a permanent, hierarchical decimal address built from dot-separated numeric segments, two digits wide by default: `00` (area), `00.01` (category), `00.01.01` (subcategory), `00.01.01.01` (context), and deeper as needed. The two-digit width is a parameter, not a fixed rule. Per-level fan-out is $10^{(\text{segment digits})}$, so widening to three or four digits lifts the ceiling from 100 children per level to 1,000 or 10,000, and the tool accepts any segment of two or more digits. The address space is per-corpus: numbering resets at every corpus root (see [docs/corpus-boundaries.md](#)). The address is permanent. Nodes retire with `deprecated: true`, never deletion.

The machine-readable manifest. `das.manifest.yaml` is the address-to-node map. An agent loads it once and can answer “where does X live?” without walking the filesystem. This is the artifact JD lacks: a single file that describes the entire address space, including optional `agent_hint` routing context primed at schema-build time.

The naming convention and type vocabulary. Filenames follow `{address}-{TAG}-{type}-{descriptor}.ext`. The type is required, drawn from a hard-capped controlled vocabulary of 17 document-intent slugs (`runbook`, `spec`, `contract`, `report`, `reference`, and so on). The address prefix on the filename, not the folder, carries the jump-table signal. A measured finding is that folder address prefixes are not required at all, which drops adoption cost to filename renames. A corpus can also declare a different naming convention in its config: declared, justified, and validated rather than left to drift (see [docs/corpus-conventions.md](#)).

Tags. An optional controlled vocabulary (codes of 2 to 5 characters: an uppercase letter, then uppercase letters and/or digits, so real product codes like `M365` qualify) for out-of-context readability, corpus-wide enumeration via `find . -name '*-TAG-*`, and in-folder disambiguation. Use tags sparingly. They help targeted enumeration but cost you on broad navigation, so the spec advises against tagging widely.

Passports. A per-document metadata container embedded in the file (for Markdown, a YAML block inside a leading HTML comment). The DAS address is the passport’s primary key. The `summary` field carries the entire RAG routing signal, and an active document has to fill it.

The validator. `das validate` walks the corpus, applies the address and filename regex rules, and cross-checks addresses against the manifest. This is the one place where the standard becomes executable, and it earns its keep in CI today.

4. The thesis, value-first

DAS makes two claims. The first is obvious. The second is the one that carries the weight.

Human-and-machine readability. One identifier serves two readers. `02.01.04` tells a person, and an LLM reading `ls` output, where they are and what sits nearby. The hierarchical prefix gives you free range queries (“everything under 02”) and lets you rule out whole irrelevant areas at a glance. The decimal look is human ergonomics layered over what is, underneath, a stable, hierarchical, lexicographically sortable, prefix-decomposable key. The novelty is not the numbering. It is the insistence that one identifier stay navigable by a human and a machine at the same time.

The deterministic floor and offline survivability: the value DAS leads with. Suppose the full agentic stack is built and the retrieval layer does most of the work. Now ask the questions that actually decide it. What happens when that layer goes offline? What happens when there is no RAG at all: air-gapped, degraded, a client who will not run the substrate, or a task you simply will not hand to a probabilistic layer? With a pure-graph or pure-RAG system, the answer is that nothing works. With DAS, the answer is that a simple agent with no RAG, or a human, can still navigate the corpus by hand: by address, by manifest, by the numeric jump-table, with the passport sitting in plain text right there in the file. The smart layer is an accelerator you have to be able to lose. DAS is the floor that keeps the corpus usable once the ceiling is gone. Graceful degradation, RAG then manifest

then filesystem, is not a fallback bolted onto the product. It is the product.

This reframes the benchmark's most-dismissed number (Section 5). The near-neutral cost of raw addresses is not the weak result it looks like. The floor is the point, and the small cost only says the floor is nearly free to keep.

Why not UUIDs. The obvious pushback is this: why hierarchical decimal addresses instead of UUIDs, or any opaque stable ID? UUIDs genuinely win on several axes, and the case has to concede them up front: no allocation coordination, no semantic baggage that can age badly, identity decoupled from location, effectively infinite space, and a machine-native fit. Those are real, and a DAS address pays for each one. But a UUID is parasitic on an external index. It means something only as long as the database or graph that maps it to meaning is alive. Lose that index and a UUID corpus is a pile of opaque-named files with no way in. A DAS address is meaningful on the filesystem itself, with no index, no database, no RAG. UUIDs have no floor. An opaque ID needs the smart layer to be navigable at all. A DAS address is navigable without it. A UUID is identity without meaning. A DAS address is identity that also encodes where the thing lives and how it is structured. They are two ends of one spectrum, and a document can carry both: an opaque ID for machine immutability, a DAS address for human navigation and placement. The bet DAS makes is narrow and falsifiable. The navigability and the offline fallback are worth the coordination and misallocation cost an opaque ID avoids. If you never lose the index and never need a human to navigate, UUIDs win. The moment either of those fails, degraded mode, an audit, an air-gap, a human in the loop, the opaque ID has nothing to offer and the address does.

There is a subtler version of the same objection: should a DAS passport carry its own internal UUID? The honest answer is "not yet, but name the day that flips," and both halves matter. A passport-internal UUID does not actually conflict with the address-is-permanent thesis. That thesis governs the filesystem identifier, where the file lives and how a human cites it. A passport UUID lives a layer down, as opaque document identity. The two coexist fine. They are not competing for the same job. The reason to reject it today is narrower and entirely practical: it has no consumer. Nothing in the standard or the tool reads it, so adding it now is a field nobody maintains and every filing agent has to remember to populate for no payoff. The trigger to adopt it is concrete: the day a `das mv` or re-address operation ships. Once an address can change, the filesystem identifier is no longer stable, and you need a stable opaque ID underneath it to keep cross-references and provenance intact across the move. That is the day the UUID earns its keep. So the posture is reject for now, with a named trigger, not reject forever.

5. Evidence, honestly framed

The evidence below is preliminary. It comes from a navigation benchmark of **56 files, a single Haiku model, three runs, with turns-to-answer as the proxy for retrieval cost**. It is happy-path and out-of-repo for the RAG variants. It is directional, not proof. The headline thesis (the resilience floor) and the resilience numbers are explicitly **not** established by this benchmark.

Measured results:

- **Descriptive naming was about +30% worse** than the addressed baseline. Removing numeric prefixes in favor of "good names" removed the jump-table and the single-read map.
- **The manifest helped: roughly -13% turns** when an agent could load the manifest as a map rather than traverse the filesystem.
- **RAG navigation over passport summaries with the mxbai embedding model was about -33% turns**, the strongest single result.
- **Embedding model dominated: mxbai routed 7 of 8 questions correctly versus nomic at 3 of 8**. The embedding model was the top RAG variable, ahead of every other knob measured.

Two cautions the data itself forces. First, the ranking quietly complicates the headline: RAG over good summaries (-33%) beat the manifest (-13%), which beat raw addresses (near neutral). On the turns-to-answer axis, the winner is RAG over good summaries, which is largely address-agnostic. That is exactly why this paper

leads with the resilience floor and not with navigation speed. Navigation speed is the axis that cheaper long context and better full-content RAG will erode, and it is not the axis DAS is built to own. Second, end-to-end answer quality is not measured with any confidence. A single noisy RAGAS pass showed low context precision, so retrieval relevance beyond turn count is not established.

What is solid today, with no caveats: **the validator works**. The address and filename rules (including variable segment width and declared per-corpus naming conventions) are executable, the manifest cross-check runs, `das validate` exits non-zero on drift, and the reference implementation carries a 180-test suite. That is the one place the standard is enforceable now.

6. Positioning and FAQ

The honest objections, each with its valid part conceded first.

Why not just folders and good file names? For a small or personal corpus, that is basically Johnny.Decimal or descriptive naming, and it is fine. But the benchmark's descriptive variant was +30% worse, because good names without numeric prefixes give you no jump-table and no single-read map, and good names drift across authors with no canonical convention and no enforcement. That is the JD-at-scale failure exactly. DAS is the disciplined, machine-checkable (`das validate`), agent-loadable (manifest) version of "folders and good names."

Why not a database, Notion, Obsidian, or a DMS? Those are more powerful for queries, relationships, and permissions, and at real scale that is where serious systems go. But they need infrastructure and a running service, and they hold your data in their format. You cannot `ls` Notion offline, grep it, diff it in git, or hand it to an air-gapped client, and if the service is down you have nothing. The no-floor problem again. DAS is the lowest common denominator every machine already has. It is not DAS or a database. DAS is the on-disk floor, and the database is the accelerator on top.

Isn't this just Johnny.Decimal with extra steps? It shares JD's permanent-numeric-address core, and the lineage is explicit. The extra steps are the point: unlimited depth (JD caps at two levels), a machine-readable manifest (JD is convention-only, and the manifest measured -13%), a controlled naming convention plus a validator (JD has neither), tags, and passports. JD was built for one person. DAS is for an organization and its agents.

Why a standard, not a product? Standards without adoption die, and a format is harder to monetize than a tool. That is a real risk, and the standard has to earn its way to necessity. But the value (legibility, the offline floor, the citable address) accrues to whoever lays out their corpus this way, no matter which tool does it. The durable asset is the format. The CLI and agents are reference implementations.

Doesn't long context plus ever-better RAG make this moot? For the happy-path retrieval task, cheaper long context and better full-content RAG do erode the navigation-speed value. That is the central risk this paper names. But it erodes the navigation value, not the floor, the provenance, the citation, or the coordination value. Better RAG is still the layer you have to be able to lose. It still has no stable citation primitive. It still cannot be audited or grepped offline. And context rot means more context is not strictly better. DAS bets on the layer RAG cannot replace.

What about access control, permissions, security? Conceded in full and stated as out of scope: DAS has no access model. It is a naming and addressing convention on a filesystem. Permissions are whatever the OS, the repo, or the layer above provides. Access control is the host system's job.

Isn't maintaining addresses and passports a huge manual burden? Assigning an address requires judgment, and a hand-maintained corpus carries a real tax. But in the agent-authored world that is the premise here, the filing agent assigns the address and writes the passport as part of ingest, and `das validate` catches drift. The force that explodes the corpus is the force that pays the maintenance tax.

Won't a single YAML manifest fall over at scale? Yes, and it is a real, already-known gap. A 20k-node manifest is a large file loaded in full, and `rglob` validation is $O(\text{corpus})$ per run. This is an engineering problem, not a thesis problem (shard the manifest, index it, or generate it from the layer above), but it is named here as

known and unsolved rather than glossed over.

Why decimal and two digits, not hex or base62? Decimal two-digit is a human-ergonomics choice, not the densest encoding, and it is a clean reject once you ask what navigation value a denser base could possibly add. The benchmark's real-v3 finding (Resolved Question 3 in the naming-convention analysis) is that folder address prefixes are not required: the filename alone carries the jump-table signal. What does the navigating, for both the human and the agent, is prefix-decomposability, the property that `02.01.04` breaks into nested ranges you can read, sort, and discard by. That property is base-agnostic. Hex has it no more than decimal does. So a denser base buys zero navigation. It can only cost you on the two axes that matter. One is legibility, which is the whole thesis: a human reads, dictates, emails, and sorts `02.01.04` with no decoder, but not `1A.0B`. The other is tokenizer noise on the embedding axis, which the benchmark named the highest-leverage RAG variable. Hex strings fragment less predictably and risk degrading the one knob that dominated every result. And if width were ever the real constraint, hex still loses: three-digit decimal strictly beats it, more legible and wider at once (1,000 children per level versus 256). Density is not the goal. Legibility is, and nothing here pays for trading it away.

What about document versioning? A real gap. DAS has a `status` field (draft / active / deprecated / archived) but no version model. This is delegated, not invented here: to the temporal layer above (supersession via valid/invalid timestamps), or to git for text. Named as deferred.

How do I migrate my existing SharePoint or Drive sprawl into this? There is no migration or bulk-ingest tool today, and the "adoption cost is just renames" claim was measured on an already-converted corpus, not a real migration. Migration is the job of the cleanup tier above DAS, the engagement that produces a DAS corpus. Until that exists, first-time adoption is manual, and that is a real cost.

Isn't this vendor lock-in to a Hexaxia format? It is a Hexaxia-authored convention, but it is the anti-lock-in choice by construction: plain folders, filenames, YAML, and Markdown on a bare filesystem, no database, no service, no proprietary store. You read it, grep it, `ls` it, and git it with zero Hexaxia software. The spec is open text. The tools are replaceable reference implementations. The only thing you are locked into is a naming convention you can walk away from by ignoring it, with every file still sitting there, still readable.

7. Limits and the validating experiment

The gaps, stated plainly:

- **No passport parsing or validation in the tool.** The summary is called the entire RAG signal throughout, yet the shipped tool does not parse, validate, or enforce passports (no address-match check, no active-requires-summary check). This is the biggest gap between spec ambition and tool reality.
- **The RAG signal is Markdown-only.** Real corpora are dominated by PDFs and DOCX, which carry no embedded passport. The standard's posture, sidecar passports for non-text formats with the plain-text sidecar as the source of truth, is documented (`docs/passport-formats.md`), but the tooling does not yet enforce artifact-sidecar pairing, and the measured results were produced on embedded Markdown passports only. The strongest measured results may not carry over to the document types that dominate real corpora.
- **No lifecycle or mutation commands.** No `deprecate`, no `mv / rename`, and no reverse validation (a manifest node whose folder was deleted passes silently).
- **A single-YAML scaling ceiling.** The manifest-in-one-read story has a context-window limit at thousands of nodes, and validation is $O(\text{corpus})$ per run.

The one experiment that would actually validate the thesis is not a turns-to-answer benchmark. It is a degraded-mode test at scale: **can a no-RAG agent, or a human, still answer real questions against a 10,000-plus document corpus using only addresses, the manifest, and passports?** That is the measurement that tests what DAS is actually for. The honest residual risk is that the fallback has to be good enough to matter at scale. If degraded-mode navigation is unusably slow or inaccurate at 10k+ documents, the insurance does not pay out. Proving the floor holds at scale is the work that would confirm everything above it.

8. Conclusion

DAS is a deterministic, human-and-machine-readable addressing layer for document corpora. Its durable claim is not that it wins retrieval. Cheaper long context and better RAG will keep pressuring the navigation-speed axis. The claim is that it provides a floor the probabilistic layers cannot: a corpus that stays navigable, citable, greppable, and auditable when the smart layer is offline, air-gapped, or untrusted. The numbering is the human-ergonomic skin over a stable hierarchical key. The durable core is dual-readable addresses plus per-document provenance, with RAG as an optional accelerator on top. The evidence so far is preliminary, and the resilience thesis is unproven at scale. The next move is to test the floor where it matters: degraded mode, at size, on a real corpus. If it holds there, DAS is the floor under the agentic stack. If it does not, DAS is a clean stepping-stone. Either way, the experiment decides it, not the assertion.

Cite as: Aaron Lamb. "Hexaxia DAS: A Deterministic, Human-and-Machine-Readable Document Addressing Standard." Hexaxia Technologies, Version 1.0, 2026.